A Lab Report
On

“Implementation of Object Oriented Concepts and Design Patterns in Java”

For The Course

“Object Oriented Pattern and Design Lab”

Course Code : ICT 2208

Submitted By

Md. Afif Zaman (IT23059)

Submitted To

Dr. Ziaur Rahman

Professor

Department of ICT, MBSTU

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902, DHAKA, BANGLADESH

1.1- Encapsulation:

- **Definition:** Encapsulation is the process of wrapping data & methods into a single class & restricting direct access to the data.

- Java achieve encapsulation by -
  1. declaring variable private
  2. using public getters
  3. using public setters
  4. applying validation when needed.

```
class student {
    public double cgpa;
    public double getCgpa(){
        return cgpa;
    }
public void setCgpa(double cgpa){
    if (cgpa >=0 && cgpa <=4.0)
        this.cgpa =cgpa;
    else
        system.out.println("Invalid CGPA");
    }
}
```

- Advantages:
  - data security
  - controlled access
  - validation.
  - easier maintenance.

- Example:

an ATM hides the internal banking logic & provides controlled buttons to the user.

1.2- polymorphism:

- Definition: polymorphism means many forms. It allows the same method name or reference to behave differently in different situations.

• Compile time polymorphism	• Runtime polymorphism
• achieved by method overloading	• achieved by method overriding
• usually within the same class	• parent-child classes.
• parameters are different.	• method signature is the same.
• decided by compiler	• decided by JVM at runtime.
• static binding	• dynamic binding.

- **Overloading Example:**

```
class calculator {
int add(int a, int b) { return a+b; }
double add (double a, double b)
  { return a+b; }
  }
```

- **Overriding example:**

```
class Animal {
    void sound() {
  system.out.println("Animal makes
                      sound");
  } }
class Dog extends Animal {
      @ override
      void sound() {
    system.out.println("Dog barks");
  } }
Animal a = new Dog();
  a.sound();
```

1.3-practical- Bank account:

```
class BankAccount {
    private double balance;
public BankAccount(double initialBalance) {
    balance = initialBalance;
public double getBalance() {
    return balance;
}
public void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
system.out.println("Deposited: "+amount);
    } else
system.out.println("Invalid amount");
    }
}
public void deposit(double amount, String remarks) {
    if (amount > 0) {
        balance += amount;
system.out.println("Deposited: "+amount);
system.out.println("Remarks: "+remarks);
    } else {
system.out.println("Invalid amount");
    }
}
public class Main {
public static void main(String[] args) {
    BankAccount acc = new BankAccount(1000);
        acc.deposit(500);
        acc.deposit(200, "Salary");
system.out.println("Final Balance: "+acc.
                    getBalance());
    }
}
```

L-2- Method overloading vs overriding :

2.1 Overloading vs overriding :

Feature	Method overloading	Method overriding
• Meaning –	• same method name with different parameters	• child class redefines parent method
• classes –	• usually same classes	• parent + child classes.
• inheritance –	• not required	• required.
• parameters –	• must be different	• must be same.
• return type –	• can't distinguish methods by return type alone	• same or covariant.
• Binding –	• compile time	• Runtime.

2.2-Early binding vs Late binding

Early binding	Late binding
• compile time	• Runtime
• mainly overloading	• mainly overriding
• method call determined by compiler	• method call determined during execution.
• static binding	• Dynamic binding.

2.3 - practical - Shape Hierarchy :

```
class Shap {
double area() { return 0; }
void describe(String name) {
system.out.println("shape:" + name);
}
void describe(String name, int sides) {
system.out.println("shape:" + name + ", sides:
" + sides);
}
}
class Circle extends Shape {
double radius;
Circle(double radius) { this.radius = radius; }
@ override
double area() {
return Math.PI * radius * radius;
}
}
class Rectangle extends Shape {
double length, width;
Rectangle(double length, double width) {
this.length = length;
this.width = width;
}
@ override
double area() {
return length * width;
}
}
```

P.T.O

```
public class Main {
public static void Main(String [] args ) {
 shape S1 = new Circle(5);
 Shape S2 = new Rectangle(4,6);
S1. describe ("Circle"); (overloading)
system.out.println("Circle Area: "+
                    S1. area());
                          (overriding)

S2. describe ("Rectangle", 4);
                    (overloading)

system.out.println("Rectangle Area:
          " + S2. area());
    }          (overriding)
}
```

L3 - Abstract Class vs interface:

3.1 Definition:

. Abstract class: A class declared with abstract that can't be instantiated directly. It may contain abstract & concrete methods, instance variables, & constructors.

. Interface: A contract that a class agrees to implement Its fields are implicitly public static final constants, modern Java also supports default & static methods.

- **Differences:**

Feature	Abstract class	interface
. Declaration	— abstract class	. interface
. Constructors	— can have	. can't have
. instance variable	— can have	. no ordinary instance fields, fields are public static final
. Methods	— abstract + concrete	. Abstract + default
. multiple inheritance	— class extends only one class	. class can implement multiple interfaces
. purpose	— common identity + shared code	. capability.

3.2 When to Chosse Which?

- Abstract class : use when related classes share common properties & implementation.
  e.g. vehicle → Car, Bike, Truck.

- Interface : use when different classes need a common capability
  e.g. Car & house can both be insurable.

3.3 - practical - Vehicle :

```
abstract class Vehicle {
    void startEngine () {
System.out.println ("Engine started");
    }
abstract String fuelType ();
    }
interface Insurable {
    double calculatepremium();
    }
class Car extends Vehicle implements
  Insurable {
    @ override
    String fuelType (){
      return " Petrol";
      }
    @ override
public double calculatepremium (){
    return 500.0;
      }
    }
```

P.T.O

```
public class Main {
public static void main(String[] args){
    Car car = new Car();
    car.startEngine();
    System.out.println("Fuel Type: " +
                        car.fuelType());
    System.out.println("premium: " +
                        car.calculatepremium
                        ());
    }
}
```

L4 - Java Collection Framework:

4.1 Arraylist vs Vector vs linked list:

Feature	Arraylist	Vector	linkedlist
• Structure	– Dynamic array	• dynamic array	• doubly linked list
• Thread safe	– No	• yes	• No
• Random access	– fast o(1)	• fast O(1)	• Slow O(n)
• Middle insert	– usually O(n)	• usually O(n)	• efficient after reaching position.
• Use	– general purpose list	• legacy synchronized list	• frequent insertion or deletion use cases.

4.2 - Set & Its Implementations:

HarshSet	linkedlist	Treeset
• No duplicates	• No duplicates	• No duplicates.
• No guaranteed order	• Insertion order	• sorted order
• generally fastest average operations	• slight extra overhead	• generally slower tree based
• Hash table	• Hash table + linked structure	• Balanced search tree.

4.3- practical - Arraylist vs TreeSet

```
import java.util.ArrayList;
import java.util.TreeSet;
public class Main {
public static void main(String[] args){
ArrayList<String> List = new ArrayList<>();
  list.add("Rafi");
  list.add("Mim");
  list.add("Zain");
  list.add("Alina");
  list.add("Karim");
system.out.println("ArrayList:");
 for (String name: list)
     system.out.println(name);
TreeSet<String> set = new TreeSet<>(list);
system.out.println("\nTreeSet:");
   for(String name: set)
       system.out.println(name);
       }
   }
```

L-5 - Multithreading & Exception Handling:

5.1 - Ways to create Threads

1. Extend Thread & override run()
2. Implement Runnable & pass it to thread.
3. use executorService for thread pool based task management.

Extending Thread	Implementing Thread	Executor Service
• Simple	• Flexible	• best for managing many tasks
• Class must extend Thread	• Class can extend another class	• uses thread pool.
• less reusable	• More reusable	• Efficient resource manage ment.

5.2- Practical - Two threads:

```
class MyThread extends Thread {
  @Override
  public void run() {
  for (int i=1; i<=5; i++)
   System.out.println("Thread 1:"+i);
      }}
class MyRunnable implements Runnable{
   @Override
  public void run() {
  for (int i=1; i<=5; i++)
  system.out.println("Thread2:"+i);
  }}
  public class Main{
public static void main(String[] args){
 MyThread t1= new MyThread();
Thread t2 = new Thread(new MyRunnable());
        t1.start();
       t2.start();
      }}
```

5.3- Custom Exception:

```
import java.util.Scanner;
class InvalidRadiusException extends Exception{
public InvalidRadiusException(String message){
    super(message);
    }}
class Circle {
public double radius;
public Circle(double radius) throws
    InvalidRadiusException {
if (radius<0)
throw new InvalidRadiusException
("Radius can't be negative");
this.radius=radius;
    }
public double area(){
return Math.PI * radius * radius;
    }}
public class Main{
public static void main(String[]args)
{
```

P.T.O

```
Scanner sc = new Scanner(System.in);
system.out.print("Enter radius: ");
  double  r = sc. nextDouble();
   try {
  circle c = new Circle (r);
system.out.println("Area: "+c.carea());
  }
 catch( InvalidRadiusException e) {
system.out.println("Error: "+e.get
                    message());
        }
    sc. close();
   }
}
```

L-6- JDBC + MVC

6.1 Steps of JDBC connection:

```
class.forName("com.mysql.cj.jdbc.Driver");
connection con =
DriverManager.getConnection
(url, user, pass);
preparedstatement ps =
con.prepareStatement(sql);
ps.executeUpdate();
ResultSet rs = ps.executeQuery();
```

6.2 MVC pattern :

MVC seperates an application into Model, View & Controller. A DAO is a separate data access component & should not simply be labeled as the MVC controller.

Component	Mainjob	Example
Model	represents data obj	student
View	user interface	JSP, HTML
Controller	hadles request & control flow	StudentServlet
DAO	performs database operations	StudentDAO

6.3 - practical - MVC student insert:

```
class student {
private int id;
private String name;
private double cgpa;
public Student (int id, String name,
                double cgpa) {
    this.id=id;
    this.name=name;
    this.cgpa=cgpa;
    }
public int getID(){ return id;}
public String getName(){ return name;}
public double getcgpa(){ return cgpa;}
    }
import java.sql.*;
class studentDAO{
private static final String URL =
"jdbc:mysql:
private static final String USER = "root";
private static final string PASS = "pass";
```

P.T.O

```
public void insert ( Student s) {
    string sql =
 " INSERT INTO Students (id, name, cgpa)
   VALUES ( ?, ?, ?)";
 try (Connection con =
   DriverManager.getConnection(URL, USER,
                               PASS);
  preparedStatement ps =
      con.prepareStatement (sql))
ps.setInt (1, s.getID());
ps.setString (2, s.getName());
ps.setdouble (3, s.getCgpa());
ps.executeUpdate ();
System.out.println("Student inserted
                    successfully.");
    }
  catch (SQLException e) {
System.out.println("Database error:
                    " + e.getMessage());
} } }
```

P.T.O

```
import java.util.Scanner;
public class Main{
public static void Main(String[] args){
Scanner sc = new Scanner(System.in);
system.out.print("Enter ID:");
  int id = sc.nextInt();
system.out.print("Enter Name:");
  String name = sc.next();
system.out.print("Enter CGPA:");
  double cgpa = sc.nextDouble();
student student = new Student(id, name,
                              cgpa);
student DAO dao = new studentDAO();
  dao.insert(student);
    sc.colse();
  }
}
```

L-7 - JavaFX House Loan Calculator :

7.1 JavaFX Application Structure :

Application : Base class for a JavaFX application.
stage : Main window.
Scene : Contains the UI content inside a stage.

GridPane: Arranges controls in rows & columns.

start() : Main JavaFX lifecycle method where the GUI is created & shown.

Structure :

Stage → Scene → Layout → Controls

7.2 & 7.3 practical - Complete loan Calculator :

```
import javafx.application.Application;
import javafx.geometry.insets;
import javafx.scene.scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.stage;
public class LoanCalculator extends Application{
private TextField loadAmountField = new TextField();
private TextField rateField = new TextField();
private TextField yearField = new TextField();

private Label monthlyLabel = new Label("
                              Monthly payment:");

private Label totalLabel = new Label("Total
                              payment:");
privateLabel interstlabel = new Label("
                              Total Interest:");

  @ override
  public void start(stage primarystage){
Grirdpane grid = new GridPane();
```

P.T.O

```
grid.setPadding(new Insets(15));
grid.setHgap(10);
grid.setVgap(10);
grid.add(new Label("Loan Amount:"), 0, 0);
grid.add(loanAmountField, 1, 0);
grid.add(new Label("LoanAmount:"), 0, 0);
grid.add(new Label("Annual Rate(%):"), 0, 1);
grid.add(rateField, 1, 1);
grid.add(new Label("number of years:"), 0, 2);
grid.add(yearsField, 1, 2);
Button button = new Button("Calculate");
grid.add(button, 1, 3);
grid.add(monthlyLabel, 0, 4, 2, 1);
grid.add(totalLabel, 0, 5, 2, 1);
grid.add(interestLabel, 0, 6, 2, 1);
button.setOnAction(e -> calculation());
Scene scene = new Scene(grid, 400, 300);
primaryStage.setTitle("House Loan
                       calculator");
primaryStage.setScene(scene);
primaryStage.show();
}
```

P.T.O

```
private void calculateLoan() {
double loan = Double.parseDouble( loan
                    AmountField.getText());
double rate = Double.parseDouble(rateField
                    .getText());
int years = Integer.parseInt(yearsField
                    .getText());
double mothlyRate = rate/100/12;
  int months = years*12;
double mothlypayment;
 if (monthlyRate == 0) {
  monthlypayment = loan/months; }
   else {
 monthly payment =
   loaan * monthlyRate *
     Math.pow(1+monthlyRate, month)
   (Math.pow(1+mothlyRate, month)/-1;
      }
double totalpayment = monthly payment * month;
double totalInterest = totalpayment - loan;
 monthlyLabel.setText
String.format("Montlypayment:%.2f",
                    monthlypayment));
```

P.T.O

```
totalLabel.setText{
String.format("Total payment :%.2f",
                totalpayment));
interestLabel.setText{
  String.format("Total, Interest :%.2f}
                 total interest ));
    }
public static void main(String [] args{
      launch(args);
        }
    }
```

L-8 - Socket programming & Java RMI !

8.1 Socket programming vs RMI

Socket programming	Java RMI
• low level communication	• high level remote obj communication
• uses sockets & streams	• Invokes methods on remote obj
• programmer manages message	• RMI handles much of the communication.
• can work with different languages	• primarily Java to Java
• Good for chat	• Good for distributed Java applications.

8.2 – Server Code :

```
import java.io.*;
import java.net.*;
public class ChatServer {
public static void main(String[] args )
  throws IOException{}
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Client connected! ");
BufferedReader in =
  new BufferedReader{
  new InputStreamReader(socket.get
                        InputStream());
PrintWriter out =
new PrinterWriter(socket.getoutput
                  stream(), true);
String message = in.readline();
System.out.println("Client says: "+message);
out.println("Message received :"+message);
   socket.close();
   serversocket.close();
      }
   }
```

8.3 – client Code

```
import java.io.*;
import java.net.*;
import java.util.Scanner;
public class ChatClient {
public static void main(String[] args) throws
        IOException {
 Socket socket = new Socket("localhost", 5000);
 PrintWriter out =
   new PrintWriter(Socket.getOutputStream(),
                   true);
 BufferedReader in =
        new BufferedReader(
     new InputStreamReader(socket.getInput
                           Stream()));
 Scanner sc = new Scanner(System.in);
  System.out.print("Enter message:");
  String message = sc.nextLine();
  out.println(message);
  String reply = in.readLine();
System.out.println("Server reply:"+reply);
        socket.close();
          sc.close();

  }
}
```

L9 - Servlet + JSP + JDBC CRUD :

9.1 project Setup :

```
CREATE DATABASE student_db;
USE student_db;
CREATE TABLE Students (
    id INT PRIMARY KEY;
    name VARCHAR(50):
    cgpa DOUBLE
);
```

Typical Flow :

Browser / JSP Form → Servlet → DAO / JDBC → MySQL Database.

L9- Servlet + JSP + JDBC CRUD:

9.2 Servlet doPost()

```
import java.io.IOException;
import java.sql.*;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.*;
public class StudentServlet extends HttpServlet {
    @Override
  protected void doPost(
    HttpServletRequest request,
    HttpServletResponse response)
    throws ServletException, IOException {
    int id =
  Integer.parseInt(request.getParameter("id"));
  String name =
  request.getParameter("name");
  double cgpa =
 Double.parseDouble(request.getParameter("cgpa"));
   String url =
   "jdbc:mysql:
      String user="root";
   String pass ="password";
   String sql =
   INSERT INTO Students(id,name,cgpa)
          VALUES(?,?,?)";
```

```
try (connection con =
 DriverManager.getConnection(url, user, pass);
 preparedStatement ps =
con.preparedStatement (sql){
ps.setInt (1, id);
 ps.setString (2, name);
ps.setDouble (3, cgpa);
 ps.executeUpdate ();
response.getWriter ().println(
" Student added successfully! ");
        }
 catch (SQLException e) {
 response.getWriter ().println(
"Database Error: " + e.getmessage());
        }
    }
}
```

9.3 - JSP - Display student records:

```
<%@ page import = "java.sql.*" %>
<html>
<head>
<title> Student Records </title>
</head>
<body>
<h2> Student Records </h2>
<table border = "1">
<tr>
    <th> ID </th>
    <th> Name </th>
    <th> CGPA </th>
</tr>

<%
String url =
"jdbc:mysql:
String user = "root";
String pass = "password";
String sql =
"SELECT id, name, cgpa FROM
    Students";
```

P.T.O

```
try (Connection con =
DriverManager.getConnection(url, user, pass);
PreparedStatement ps =
 con.preparedStatement (sql);
 ResultSet rs =
  ps.executeQuery ()) {
   while (rs.next()) {
    %>
<tr>
  <td><%= rs.getInt("id") %></td>
  <td><%= rs.getString("name") %></td>
<td><%= rs.getDouble("cgpa") %></td>
   </tr>
    <% }
     } catch (SQLException e) {
    %>
   <tr>
    <td colspan = "3">
   Database Error : <%= e.getMessage() %>
     </td>
   </tr>
   <% }
   %>
  </table>
  </body>
  </html>
```

10.1 Steps to set up Spring Boot + REST + JPA (Maven deps: spring-boot-starter-web, spring-boot-starter-data-jpa, MySQL connector) and role of embedded Tomcat.

Solution:

Spring Boot + REST + JPA Setup

1. Create Spring Boot Project

- Create a Maven Spring Boot project.
- Select Java and the required Spring Boot version.
- Configure Group, Artifact, and Java version.

2. Add Maven Dependencies

Add these dependencies in pom.xml**:**

```
<dependencies>
   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
   </dependency>
   <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
   </dependency>
</dependencies>
```

3. Configure MySQL

In application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
```

spring.datasource.password=1234

spring.jpa.hibernate.ddl-auto=update

4. Create JPA Entity

Create a class such as **Student** and use @Entity.

- Entity → Represents database table
- JPA → Maps Java objects to database records

5. Create Repository

Create an interface extending JpaRepository.

- It provides common operations such as save, find, update, and delete without writing SQL manually.

6. Create REST Controller

Use @RestController to create REST API endpoints.

Example:

```
@RestController
class StudentController {


   @GetMapping("/students")
   public String students() {
      return "Student List";
   }
}
```

7. Embedded Tomcat

- spring-boot-starter-web includes embedded Tomcat by default.
- It allows the Spring Boot application to run directly without installing/configuring an external Tomcat server.

10.2 JPA Student entity (id, name, cgpa) + StudentRepository extends JpaRepository.

Solution:

1.Student Entity:

```
import jakarta.persistence.*;
```

```
@Entity
public class Student {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;
   private String name;
   private double cgpa;
   public Student() {}
}
```

- @Entity → Maps the class to a database table.
- @Id → Defines id as the primary key.
- @GeneratedValue → Automatically generates the ID.

2. StudentRepository

```
import org.springframework.data.jpa.repository.JpaRepository;
public interface StudentRepository
      extends JpaRepository<Student, Integer> {
}
```

- JpaRepository provides built-in methods such as save(), findAll(), findById(), and deleteById().
- Therefore, we do not need to write basic SQL queries manually.

Structure

Student Entity → StudentRepository → MySQL Database

10.3 @RestController with GET /students and POST /students endpoints using the repository.

Solution :

```
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/students")
public class StudentController {
   private final StudentRepository repo;
```

```
    public StudentController(StudentRepository repo) {
        this.repo = repo;
    }
    // GET /students
    @GetMapping
    public List<Student> getStudents() {
        return repo.findAll();
    }
    @PostMapping
    public Student addStudent(@RequestBody Student student) {
        return repo.save(student);
    }
}
```

Key Points:

- **@RestController** → Makes the class a REST API controller.
- **@GetMapping** → Handles GET /students and returns all students.
- **@PostMapping** → Handles POST /students and adds a new student.
- **@RequestBody** → Converts JSON request data into a Student object.
- **repo.findAll()** → Retrieves all students

11.1 Design DB schema for quiz on Crops / Geography / Academic Institutions of your district + Player/Score table. Justify design.

Solution:

Quiz Database Schema

For a quiz on Crops, Geography, and Academic Institutions of a district, we can design the database using separate tables for questions, players, and scores.

1. Questions Table

Field	Type	Description
question_id	INT, PK	Unique question ID
question_text	VARCHAR	Question
option_a	VARCHAR	Option A
option_b	VARCHAR	Option B
option_c	VARCHAR	Option C
option_d	VARCHAR	Option D
correct_option	CHAR	Correct answer

2. Players Table

Field	Type	Description
player_id	INT, PK	Unique player ID
player_name	VARCHAR	Player name

3. Scores Table

Field	Type	Description
score_id	INT, PK	Unique score ID
player_id	INT, FK	References Players
score	INT	Obtained score

Field	Type	Description
quiz_date	DATE	Date of quiz

SQL Schema

```
CREATE DATABASE district_quiz;
USE district_quiz;
CREATE TABLE Questions (
    question_id INT PRIMARY KEY AUTO_INCREMENT,
    question_text VARCHAR(255),
    option_a VARCHAR(100),
    option_b VARCHAR(100),
    option_c VARCHAR(100),
    option_d VARCHAR(100),
    correct_option CHAR(1),
    category VARCHAR(50)
);
CREATE TABLE Players (
    player_id INT PRIMARY KEY AUTO_INCREMENT,
    player_name VARCHAR(100)
);
CREATE TABLE Scores (
    score_id INT PRIMARY KEY AUTO_INCREMENT,
    player_id INT,
    score INT,
    quiz_date DATE,
    FOREIGN KEY (player_id) REFERENCES Players(player_id)
);
```

Justification

 Separate category field: Allows questions to be grouped into Crops, Geography, and Academic Institutions.

 Players and Scores are separate: One player can have multiple quiz scores**.**

 Primary Keys: Ensure every question, player, and score has a unique identity**.**

 Foreign Key: Scores.player_id connects each score to the correct player**.**

 The design reduces data duplication and makes searching, updating, and generating leaderboards easier.

11.2 Servlet method: save Player name + final score into PlayerScore via JDBC/PreparedStatement.

Solution: Servlet Method to Save Player Score : The Servlet receives the Player Name and Final Score and stores them in the PlayerScore table using JDBC and PreparedStatement.

```
protected void doPost(HttpServletRequest request,
                HttpServletResponse response)
                throws ServletException, IOException {
   String name = request.getParameter("name");
   int score = Integer.parseInt(request.getParameter("score"));
   String url = "jdbc:mysql://localhost:3306/district_quiz";
   String user = "root";
   String password = "1234";
   String sql = "INSERT INTO PlayerScore(player_name, score) VALUES (?, ?)";
   try {
      Connection con = DriverManager.getConnection(url, user, password);
      PreparedStatement ps = con.prepareStatement(sql);
      ps.setString(1, name);
      ps.setInt(2, score);
      ps.executeUpdate();
      response.getWriter().println("Score saved successfully.");
```

```
        ps.close();
        con.close();
    } catch (SQLException e) {
        response.getWriter().println("Database Error: " + e.getMessage());
    }
}
```

Key Points

- **getParameter()** reads the player's name and final score.
- **PreparedStatement** safely inserts the data.
- **executeUpdate()** saves the record in PlayerScore.
- **try-catch** handles database errors.

11.3 Quiz logic: ≥3 MCQs (crop, geography, institution) as objects; present, check answer, increment score.

Solution :

Quiz Logic Using Java Objects

```
class Question {
     String question, a, b, c, d, correct;
    Question(String q, String a, String b, String c,
            String d, String correct) {
        this.question = q;
        this.a = a; this.b = b; this.c = c;
        this.d = d; this.correct = correct;
    }
}
public class Quiz {
    public static void main(String[] args) {
```

```
        Question[] questions = {
            new Question(
                "Which is a major crop of Bangladesh?",
                "Rice", "Tea", "Coffee", "Cocoa", "A"),
            new Question(
                "Which district is famous for Mahasthangarh?",
                "Bogura", "Dhaka", "Khulna", "Sylhet", "A"),
            new Question(
                "Which is an academic institution?",
                "MBSTU", "Padma River", "Rice", "Hilsa", "A")
        };
        int score = 0;
        for (Question q : questions) {
            System.out.println(q.question);
            System.out.println("A. " + q.a);
            System.out.println("B. " + q.b);
            System.out.println("C. " + q.c);
            System.out.println("D. " + q.d);
            String answer = "A";
            if (answer.equalsIgnoreCase(q.correct))
                score++;
        }
        System.out.println("Final Score: " + score);
    }
}
```

12.1 List all 5 GoF Creational patterns (Singleton, Factory Method, Abstract Factory, Builder, Prototype) with one-line purpose each.

Solution : 5 GoF Creational Design Patterns

Creational patterns deal with how objects are created.

1. **Singleton** — Ensures that a class has only one object/instance and provides a global access point to it.
2. **Factory Method** — Defines a method for creating objects while allowing subclasses to decide which object to create.
3. **Abstract Factory** — Provides an interface for creating families of related objects without specifying their concrete classes.
4. **Builder** — Builds a complex object step-by-step, allowing different representations of the same object.
5. **Prototype** — Creates new objects by copying/cloning an existing object.

12.2 List all 7 GoF Structural patterns (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) with one-line purpose each.

Solution : 7 GoF Structural Design Patterns

Structural patterns deal with how classes and objects are combined to form larger structures.

1. **Adapter** — Converts the interface of one class into another interface that the client expects.
2. **Bridge** — Separates an abstraction from its implementation so both can vary independently.
3. **Composite** — Treats individual objects and groups of objects uniformly using a tree-like structure.
4. **Decorator —** Adds new responsibilities or features to an object dynamically without changing its class.
5. **Facade —** Provides a simple interface to a complex subsystem.
6. **Flyweight —** Reduces memory usage by sharing common objects/data among many objects.
7. **Proxy —** Provides a substitute or placeholder for another object to control access to it.

12.3 Implement Singleton (thread-safe) and Adapter with short, complete Java code examples.

Solution :

1. **Singleton Pattern — Thread-Safe**

Purpose: Ensures that a class has only one instance throughout the application.

```
class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null)
                    instance = new Singleton();
            }
        }
        return instance;
    }
}
public class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();
        System.out.println(s1 == s2);   // true
    }
}
```

Why Thread-Safe?

- **volatile** ensures visibility between threads.

- **synchronized** prevents multiple threads from creating multiple objects.
- **Double-checked locking** improves performance by avoiding synchronization after the instance is created.

2. Adapter Pattern

Purpose: Allows **incompatible interfaces** to work together.

```
// Target interface
interface Printer {
    void print(String text);
}
// Existing incompatible class
class OldPrinter {
    void oldPrint(String text) {
        System.out.println(text);
    }
}
// Adapter
class PrinterAdapter implements Printer {
    private OldPrinter printer = new OldPrinter();

    public void print(String text) {
        printer.oldPrint(text);
    }
}

// Client
public class Main {
    public static void main(String[] args) {
```

```
        Printer p = new PrinterAdapter();
        p.print("Hello World");
    }
}
```

Adapter Working

Client → Printer → PrinterAdapter → OldPrinter

- **Printer** → Expected interface.
- **OldPrinter** → Existing incompatible class.
- **PrinterAdapter** → Converts the old interface to the required interface.